

Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring

Year:2026

Question:

Assignment 15: Write a multithreaded C program using POSIX threads (pthread) where multiple threads increment a shared global counter variable many times. First implement the program without any synchronization and observe the incorrect output caused by a race condition. Then modify the program using a mutex (pthread_mutex_t) to protect the critical section and produce the correct final counter value. Your program must demonstrate thread creation using pthread_create(), synchronization using pthread_mutex_lock() and pthread_mutex_unlock(), and thread completion using pthread_join(). Also explain briefly why the race condition occurs and how the mutex solves the problem.

Answer:

Without synchronization:

```
#include <stdio.h>
```

```
#include <windows.h>
```

```
#define NUM_THREADS 4
```

```
#define INCREMENTS 1000000
```

```
int counter = 0;
```

```
void *increment_counter(void *arg)
```

```
{
```

```
    for(int i = 0; i < INCREMENTS; i++)
```

```
{
    counter++;
}

return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

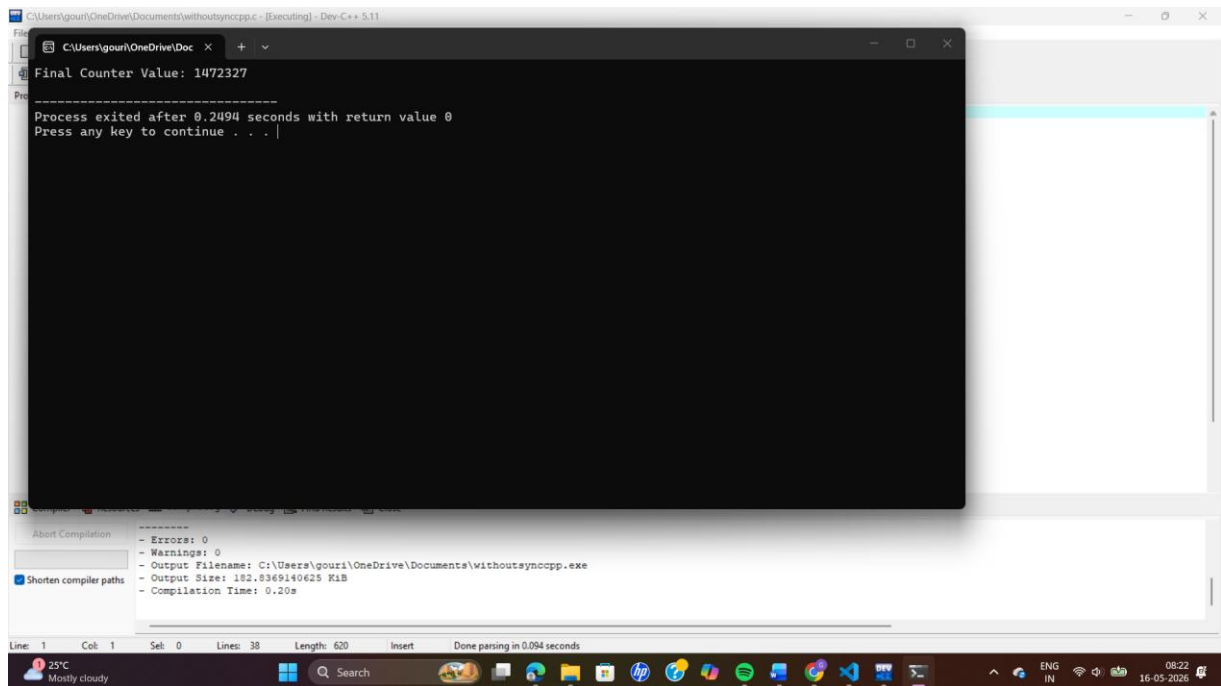
    for(int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    for(int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Final Counter Value: %d\n", counter);

    return 0;
}
```

Screenshots of the output:



Mutex synchronization:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#define NUM_THREADS 4
```

```
#define INCREMENTS 1000000
```

```
int counter = 0;
```

```
pthread_mutex_t lock;
```

```
void *increment_counter(void *arg)
```

```
{
```

```
for(int i = 0; i < INCREMENTS; i++)
{
    pthread_mutex_lock(&lock);

    counter++;

    pthread_mutex_unlock(&lock);
}

return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);

    for(int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    for(int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);

    printf("Final Counter Value: %d\n", counter);
}
```

```
return 0;  
}
```

Screenshots of the output:

